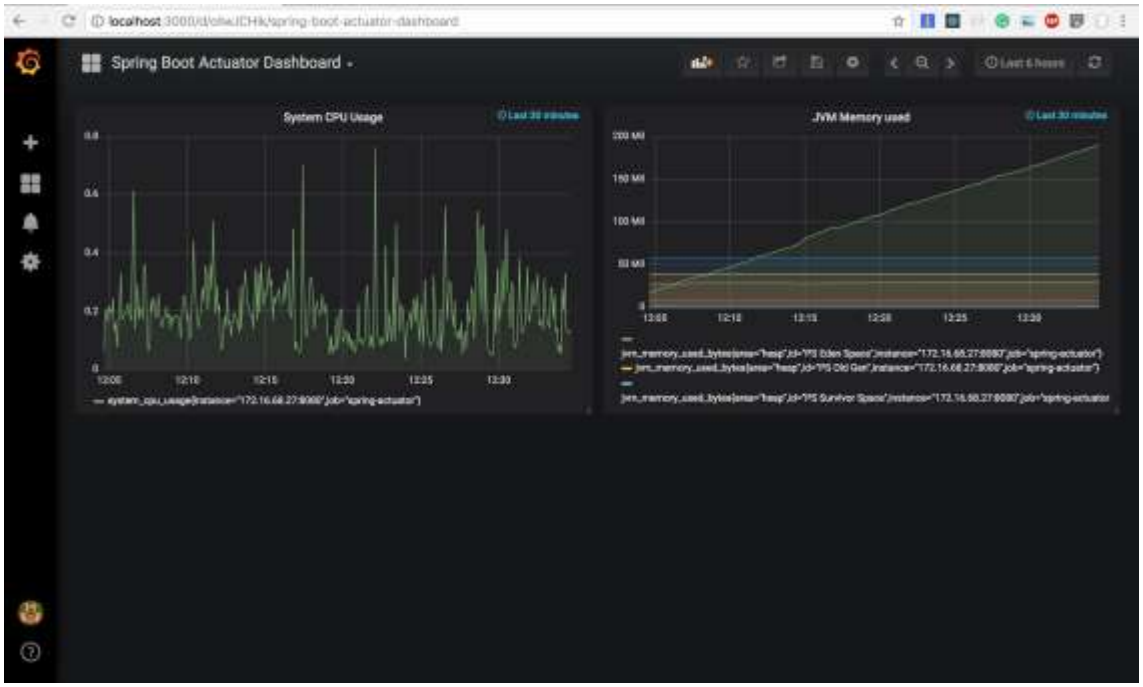
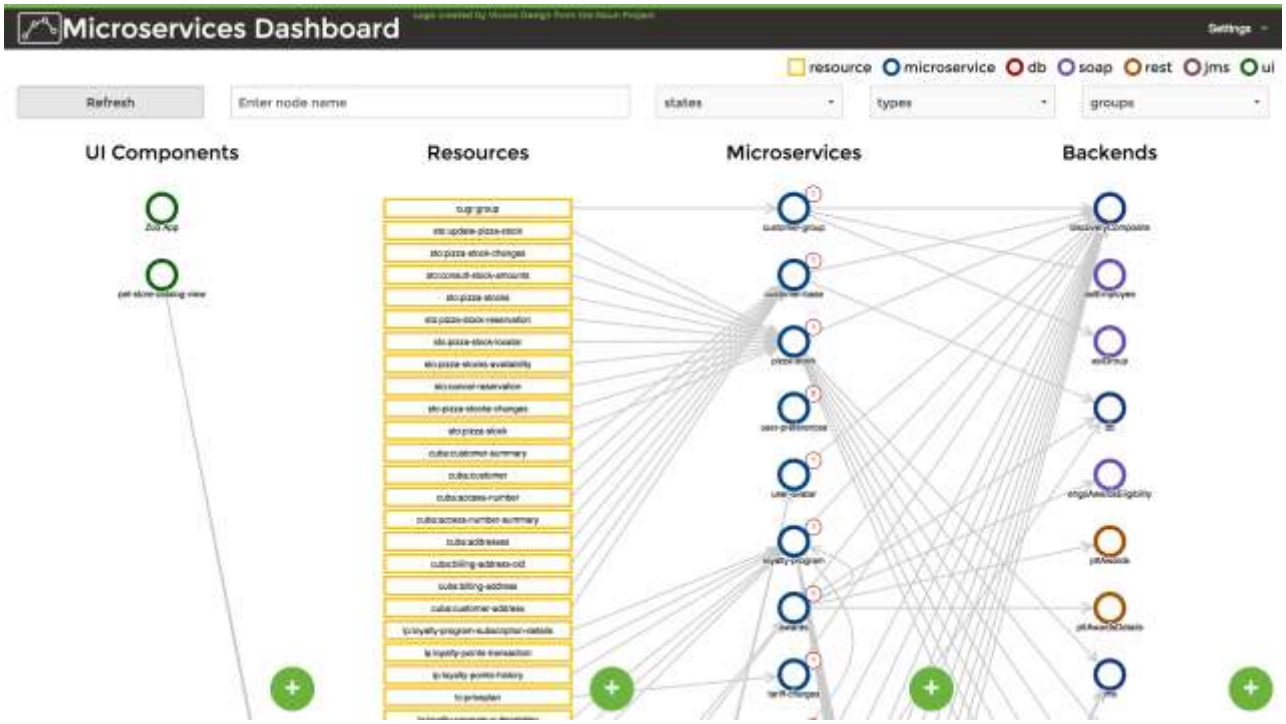
In **real-world Spring Boot applications**, the **Actuator** is used to **monitor, manage, and troubleshoot running applications in production** without stopping the system.

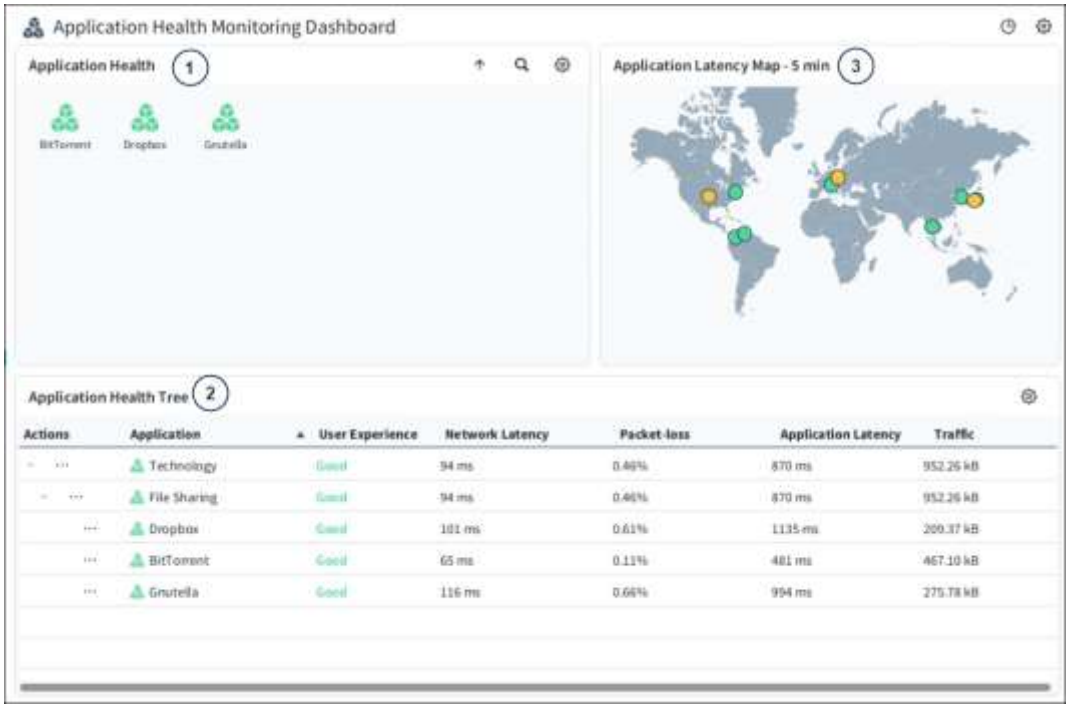
It provides **built-in endpoints** that show the **health, metrics, configuration, and internal state** of the application.

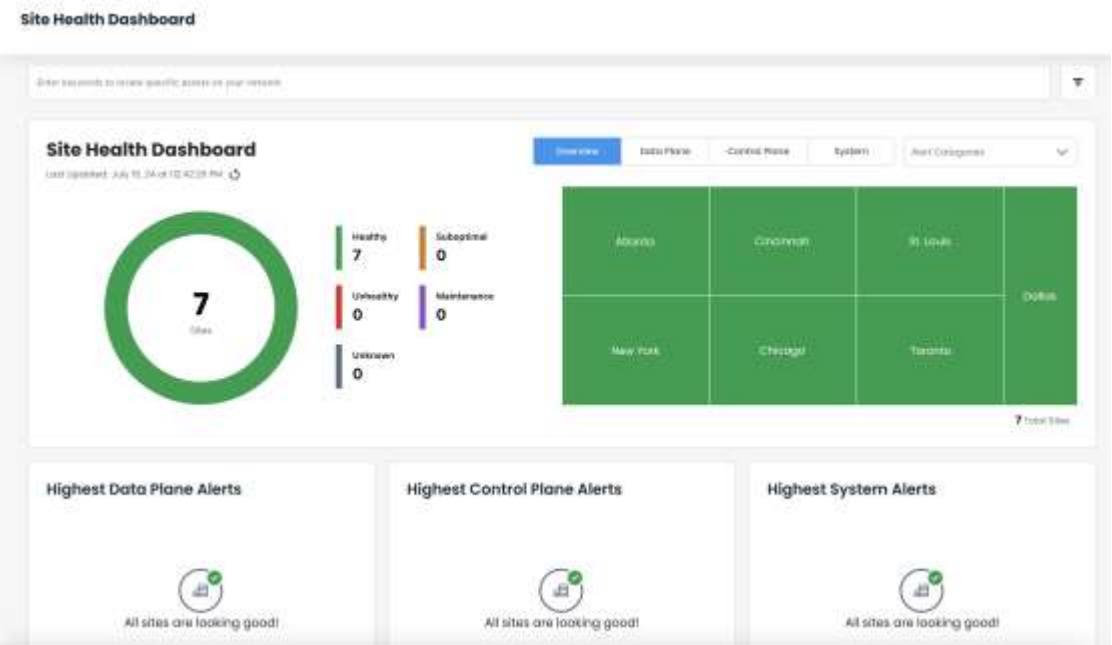
---

1 Application Health Monitoring




Microservices Dashboard
Settings
resource
microservice
db
soap
rest
jms
ui
Refresh
Enter node name
states
types
groups
UI Components
Resources
Microservices
Backends


Application Health Monitoring Dashboard
Application Health
1
BitTorrent
Dropbox
Gnutella
Application Latency Map - 5 min
3
Application Health Tree
2
Actions
Application
User Experience
Network Latency
Packet-loss
Application Latency
Traffic
Technology
Good
94 ms
0.46%
870 ms
952.26 kB
File Sharing
Good
94 ms
0.46%
870 ms
952.26 kB
Dropbox
Good
101 ms
0.61%
1135 ms
200.37 kB
BitTorrent
Good
65 ms
0.11%
481 ms
467.10 kB
Gnutella
Good
116 ms
0.66%
994 ms
275.78 kB



The **/actuator/health** endpoint tells whether the application is running properly.

Example:

http://localhost:8080/actuator/health

Response:

```
{
  "status": "UP"
}
```

Real-time use

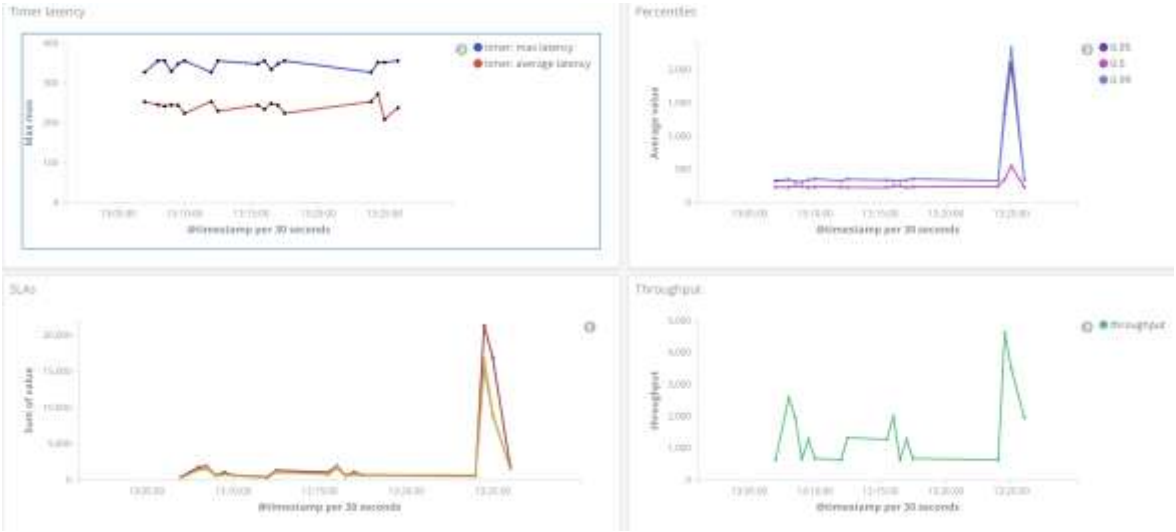
DevOps tools like **Kubernetes, Docker, AWS, Jenkins** use this endpoint to check if the application is alive.

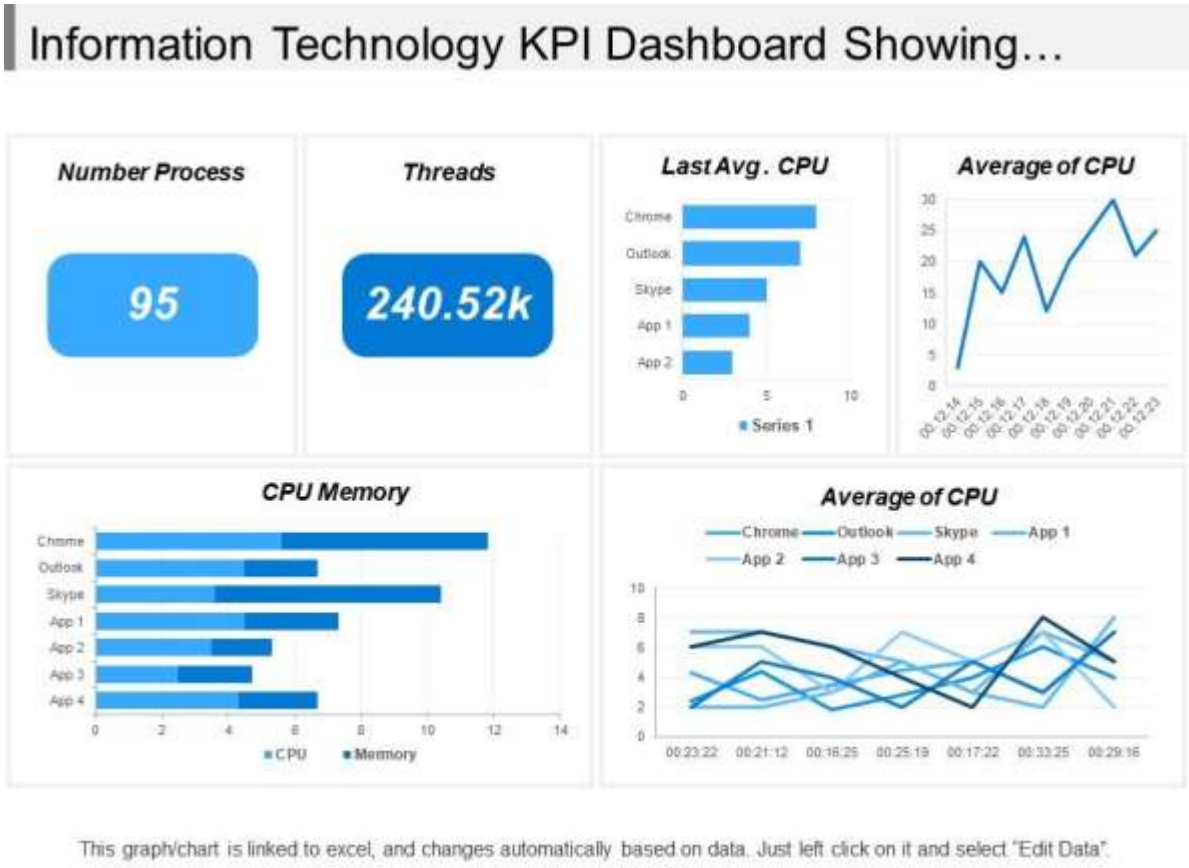
Example scenario:

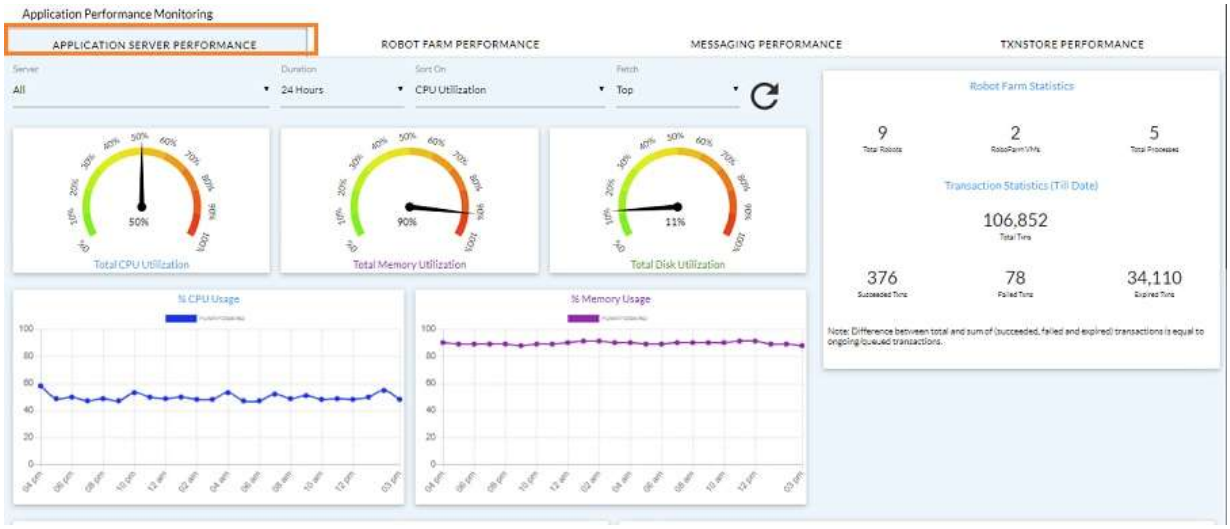
- If the health becomes **DOWN**
- Kubernetes automatically **restarts the container**

---

2 Application Metrics Monitoring







The **/actuator/metrics** endpoint provides runtime metrics like:

- CPU usage
- Memory usage
- HTTP request count
- Database connection pool usage

Example:

/actuator/metrics/http.server.requests

Real-time use

Monitoring tools like:

- **Prometheus**
- **Grafana**
- **New Relic**
- **Datadog**

collect these metrics to monitor system performance.

Example:

Requests per second
Average response time
Error rate

---

3 Debugging Production Issues

Developers use actuator endpoints to **debug problems without restarting the server**.

Useful endpoints:

Endpoint	Use
/actuator/beans	View all Spring beans
/actuator/env	Check environment variables
/actuator/mappings	See all API mappings
/actuator/loggers	Change log level at runtime

Example:

/actuator/mappings

Shows all controller endpoints.

---

4 Runtime Log Level Change

Example:

POST /actuator/loggers/com.example

Change log level dynamically:

```
{
 "configuredLevel": "DEBUG"
}
```

Real-time use

If production issue occurs:

Instead of redeploying the application
You can enable **DEBUG logs instantly**

5Database & Dependency Health Check

Actuator checks dependencies like:

- Database
- Disk space
- Redis
- Kafka

Example:

```
{
 "status": "UP",
 "components": {
   "db": { "status": "UP" },
   "diskSpace": { "status": "UP" }
 }
}
```

Real-time use

If database goes down → system alerts DevOps team.

---

6 Application Configuration Visibility

Endpoint:

/actuator/env

Shows:

- application.properties values
- system properties
- environment variables

Real-time use

Helps debug **wrong configuration issues in production**.

7Cloud & Microservices Monitoring

In **microservices architecture**, actuator is essential.

Example monitoring flow:

```
Spring Boot App
    ↓
Actuator Metrics
    ↓
Micrometer
    ↓
Prometheus
    ↓
Grafana Dashboard
```

This allows **live monitoring of hundreds of microservices**.

---

8 Real Production Example

Imagine an **e-commerce system**.

If users complain the site is slow:

DevOps checks:

/actuator/metrics/http.server.requests

They may discover:

Average response time = 5 seconds

Then they analyze:

- database connection pool
- memory usage
- thread usage

All through actuator.

---

9 Summary (Real Use)

Actuator is used in production for:

Purpose	Example
Health check	/actuator/health
Performance monitoring	/actuator/metrics
Debugging	/actuator/beans
Runtime log control	/actuator/loggers
Configuration inspection	/actuator/env
API inspection	/actuator/mappings

In simple words

Actuator = **Monitoring + Management tool for Spring Boot applications in production**.